

Bachelor AI - MLOps

08 MODEL PLATFORMS, REGISTRY, MLFLOW

Tobias Mérinat

`tobias.merinat@hslu.ch`

HSLU Hochschule
Luzern

DEPARTMENT OF INFORMATION TECHNOLOGY
Lucerne University of Applied Sciences and Arts
6343 Rotkreuz, Switzerland

September 17/24 and October 15 2026

The concept of *Model Platform* describes how Machine Learning models are packaged, stored, managed, and used. A Model Platform consists of three main components:

- the Model Deployment API
- the Model Registry, and
- the Model Server (also Prediction Server/Service)

Tracking experiments with different models and hyperparameters during the development phase can be considered a fourth part of the model platform.

Experiment Tracking serves on the one hand to avoid chaos during development, and on the other hand aims at the reproducibility of the finally used models.

The Deployment API describes how fully trained models are packaged. Models must be serialized including trained parameters, they must be versioned, and must specify or package all dependencies.

The Open Source tool **MLFlow**, which will be introduced shortly, defines such a Deployment API. **ONNX** (Open Neural Network Exchange), which is an Open Source format for Neural Networks, is another example of a Deployment API.

The task of the *Model Registry* (also referred to as *Model Store*) is to store versioned Machine Learning models, manage them, and make them available for prediction.

In addition to the models (artifacts), the Registry stores metadata such as the parameters used in training, dependencies, and performance metrics.

The Registry is a central point to enable traceability, reproducibility, and automated deployment.

A Model Registry consists of the following components:

- An Artifact Store, usually an Object Store, to save models
- A database for storing metadata
- A GUI for managing models
- An API for retrieving models and metadata

A Model Registry should store the following information:

- The complete environment (dependencies and Python version)
- A reference to the training data and commit hashes of the entire Feature Engineering and Training code
- Hyperparameters
- Performance metrics
- The executable model artifact for deployment

- Experiment Tracking is used to
 - maintain overview during the development process
 - easily compare experiments (the candidate models)
 - reproduce the relevant models from the development phase
- The Model Registry is needed
 - to reproducibly transition models into productive operation
 - to annotate models as needed
 - to represent the model lifecycle during production

The *Model Server* (also *Prediction Server*, *Inference Server*) is responsible for fetching models from the Registry, allocating resources accordingly, and providing a REST or gRPC endpoint for prediction requests.

The range of Model Servers is broad. Some better-known Model Servers are **MLFlow**, **Triton Inference Server**, **Seldon ML Server**, **Kserve**, **BentoML**, **TensorFlow Serving**, **TorchServe**, **Multi Model Server**, and **OpenVINO Model Server**.

Model Servers can be specialized for specific architectures (e.g., TensorFlow Serving, TorchServe) or support multiple Model Deployment APIs (Triton, MLFlow, Seldon, KServe).

Machine Learning models can be used or deployed in various ways:

- Multiple models/model versions in a central Inference Server
- Each model together with a REST/gRPC API in its own container
- Directly integrated into a business application
- Integrated into a batch or streaming pipeline

Each variant has advantages and disadvantages.

There are several things to consider when choosing a deployment variant:

- Decoupling of model and application
- How easy is it to update a model
- Can multiple versions of a model be used simultaneously
- Is it better to load a model into memory or execute it via an API?
- When a model is encapsulated in a container, should the model be compiled into the container or loaded over the network after container startup?

Within the course, the OpenSource software **MLFlow** is used. MLFlow covers some requirements of MLOps and allows us to try out a lot ourselves.

However, MLFlow is by no means the only tool and doesn't have to be the first choice for a *Model Platform*. It is opinionated, the documentation is comprehensive but also quite chaotic, and MLFlow's functionality leaves something to be desired in some areas.

The course is not about providing a deep introduction to MLFlow as a tool, but rather about conveying the concepts that MLOps implements more or less well.

MLFlow consists of a series of components. We use the following three:

- **Tracking:** An API to log and visualize information that arises during the training of models
- **Models:** A standard format for packaging Machine Learning models
- **Model Registry:** A service to store models, make them accessible, and manage their lifecycle

MLFlow offers the following additional components, which we do not use here

- **Projects:** Conventions to improve the reproducibility of ML projects
- **Dataset Tracking:** A submodule of *Tracking* to log datasets
- **LLM:** Functionality to interact with Cloud LLM APIs
- **Model Serving:** A way to package an ML model and a REST API into a container.

MLFlow defines a **package format** for ML models to simplify their use for inference. It includes

- A folder structure as storage format
- A metadata file named *MLModel*
- A **list of dependencies**

MLFlow Models: Folder Structure of an Sklearn model

Folder structure as written by `mlflow.sklearn.save_model()`:

```
my_model/  
|-- MLmodel  
|-- model.pkl  
|-- conda.yaml  
|-- python_env.yaml  
__ requirements.txt
```

Here is an *MLModel* file that defines a model in two variants (*flavors* in MLFlow language):

```
time_created: 2018-05-25T17:28:53.35
```

```
flavors:
```

```
  sklearn:
```

```
    sklearn_version: 0.19.1
```

```
    pickled_model: model.pkl
```

```
python_function:
```

```
  loader_module: mlflow.sklearn
```

Many Model Flavors are supported by MLFlow (Sklearn, PyTorch, Keras, ONNX, XGBoost, Spacy, . . .). A generic 'pyfunc' flavor is also available.

MLFlow Models: Signature and Model Input Example

When logging a model, a *Model Signature* and/or an *Input Example* can be passed. Signatures define model input, output, and parameters for inference.

This way, corresponding schemas can be enforced, which on the one hand better documents the model and makes its use easier, and on the other hand simplifies the detection of data anomalies in operation.

MLFlow Model Signatures are only one way to do syntactic data validation, another well-known tool is **Great Expectations**.

MLFlow Tracking consists of the following components:

- **Tracking Server:**

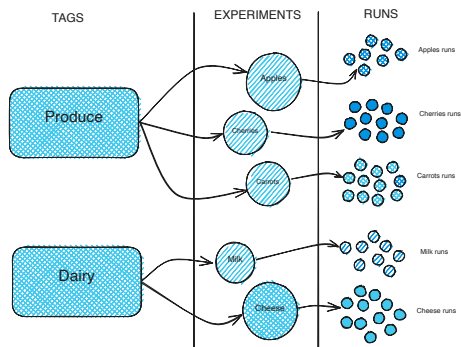
- REST API
- Web UI

- **Metadata Store:** File-Based or SQL Database

- **Artifact Store:** For storing large files (Filesystem or S3)

Experiments contain *Runs*, which in turn can contain *Child-Runs*. Further organization is possible via freely assignable *Tags*.

- **Experiment:** Everything that uses the same input
- **Run:** One Training-Run (with the idea that different runs un an experiment are comparable)
- **Child-Run:** Used for Hyperparameter-Sweeps



MLFlow Tracking: Logging Setup (1)

To use the MLFlow API, either the Tracking URI or the environment variable `MLFLOW_TRACKING_URI` must be set:

```
mlflow.set_tracking_uri("http://127.0.0.1:8080")
```

If **authentication** is enabled, the environment variables `MLFLOW_TRACKING_USERNAME` and `MLFLOW_TRACKING_PASSWORD` must also be set or these values must be passed via a credentials file.

MLFlow Tracking: Set Experiment (2)

Next, an experiment name should be set. The return value, which is not mandatory to assign, contains metadata about the experiment.

```
experiment_metadata = mlflow.set_experiment("Apple_Models")
```

If this step is omitted, MLFlow assigns the subsequent runs to a *Default Experiment*. The default experiment should not be used; it only serves as a catch-all so that no runs are lost if the experiment was not set.

MLFlow Tracking: Perform Training) (3)

Now a model is trained, in this example with Scikit-Learn. This step does not involve MLFlow-specific code. The preparation of the datasets is not shown here.

```
1 params = {  
2     "n_estimators": 100,  
3     "max_depth": 3,  
4 }  
5 rf = RandomForestClassifier(**params)  
6 rf.fit(X_train, y_train)  
7 y_pred = rf.predict(X_val)  
8 metrics = { "acc": accuracy_score(y_val, y_pred) }
```

The model signature can be derived from the data:

```
9  from mlflow.models import infer_signature
10 signature = infer_signature(model_input=X_train,
11                             model_output=y_train)
```

MLFlow Tracking: Manual Logging (5)

And finally the results are sent to the Tracking Server.

```
11 with mlflow.start_run(run_name="some run-specific name") as run:
12     mlflow.log_params(params)
13     mlflow.log_metrics(metrics)
14     mlflow.sklearn.log_model(
15         sk_model=rf, signature=signature,
16         name="relative/to/artifact/root"
17     )
```

MLFlow Tracking: Automatic Logging

For a variety of ML libraries (Sklearn, PyTorch, LightGBM, . . .), logging can also be done automatically.

```
mlflow.set_experiment("Apple_Models")
mlflow.autolog()  # must be activated BEFORE metrics import
from sklearn.metrics import accuracy_score
rf = RandomForestClassifier(n_estimators=100, max_depth=3)
# ONLY logs fit(), fit_transform(), fit_predict()
# and not other functions like crossvalscore()
rf.fit(X_train, y_train)
```

In this process, metrics, parameters, artifacts, and the model signature are logged automatically.

MLflow Tracking: UI Overview

mlflow 2.14.1 Experiments Models 🔍 ⚙️ [GitHub](#) [Docs](#)

Experiments

☐

 Default ✎ 🗑

☒ MLflow Example ✎ 🗑

MLflow Example

📄 Provide Feedback 🔗 Add Description 🔗 Share

📄 ⋮ 🔄 + New run

Time created ▾

State: Active ▾

Datasets ▾

⌵ Sort: Created ▾

📄 Columns ▾

📄 Group by ▾

Table

Chart

Evaluation

Experimental

Traces

Experimental

☐	Run Name	Created	Dataset
☐	🔴 upset-bug-224	✅ 26 seconds ago	-

MLFlow Tracking: Run View (1/2)

[MLflow Example](#) >

upset-bug-224

⋮ [Model registered](#) [↗](#)

Overview Model metrics System metrics Artifacts

Description [✎](#)

No description

Details

Created at	2024-07-25 16:36:36
Created by	root
Experiment ID	1 ↗
Status	✅ Finished
Run ID	7180c89f925946b0b03338663640810d ↗
Duration	4.1s
Datasets used	—
Tags	Training Info: Basic RF classifier model for iris ... ✎
Source	🏠 ipykernel_launcher.py

MLFlow Tracking: Run View (2/2)

Tags	Training Info: Basic RF classifier model for iris ... 
Source	 ipykernel_launcher.py
Logged models	 sklearn
Registered models	 tracking-quickstart v1

Parameters (2)

 Search parameters	
Parameter	Value
n_estimators	100
max_depth	3

Metrics (1)

 Search metrics	
Metric	Value
acc	1

MLFlow Tracking: Second Run

mlflow 2.14.1 Experiments Models 🔍 ⚙️ GitHub Docs

Experiments

Search Experiments

☐

 Default ✎ 🗑

☒ MLflow Example ✎ 🗑

MLflow Example

ℹ️ [Provide Feedback](#) [Add Description](#) Share

ℹ️ ⋮ 🔄 + New run

Time created ▾

State: Active ▾

Datasets ▾

⌵ Sort: Created ▾

📊 Columns ▾

📅 Group by ▾

Table

Chart

Evaluation

Experimental

Traces

Experimental

☐	Run Name	Created	⌵	Dataset
☐	🔴 second run	✅ 7 seconds ago		-
☐	🔴 upset-bug-224	✅ 8 minutes ago		-

MLflow Tracking: Compare Runs

MLflow Example ⓘ [Provide Feedback](#) [Add Description](#)

Share

metrics.rmse < 1 and params.model = "tree" ⓘ

Time created ▾

State: Active ▾

Datasets ▾

Sort: Created ▾

⋮

↺

+ New run

Group by ▾

Table

Chart

Evaluation

Experimental

Traces

Experimental

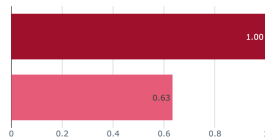
☐	☐	Run Name
☐	☒	second run
☐	☒	upset-bug-224

Search metric charts

Model metrics (2)

+ Add chart

acc



second run upset-bug-224

accuracy



second run upset-bug-224

System metrics (0)

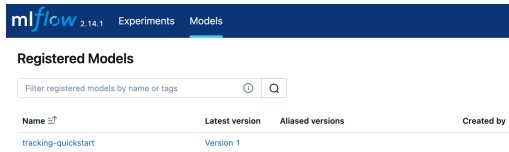
+ Add chart



MLFlow Registry: Getting Started

In the *Model Registry*, models are maintained and managed. It allows the use, storage, deletion, versioning, tagging, and annotation of models.

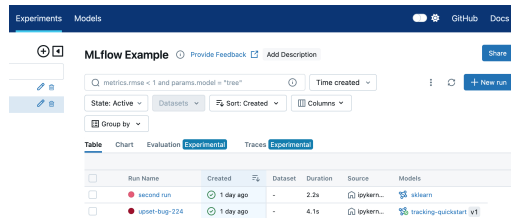
The Model Registry can be accessed via the *Model* link in the header.



MLFlow Registry: Register Models

Not every logged model needs to be registered. Models can either be registered directly during logging via an optional parameter from `mlflow.*.log_model()`, or they can be registered later via UI or API.

`mlflow.autolog()` does not register the logged models.



Run Name	Created	Dataset	Duration	Source	Models
second run	1 day ago	-	2.2s	ipykern...	sklearn
upset-bug-224	1 day ago	-	4.1s	ipykern...	tracking-quickstart v1

Registered models can receive an optional description, can be tagged, and are versioned.

Registered Models >

tracking-quickstart

Created Time: 2024-07-25 16:36:40

Last Modified: 2024-07-27 08:07:17

▼ Description

Edit

Coolset Model Ever!

▼ Tags

Name	Value	Actions
purpose	teaching	 

Name

Value

Add

▼ Versions

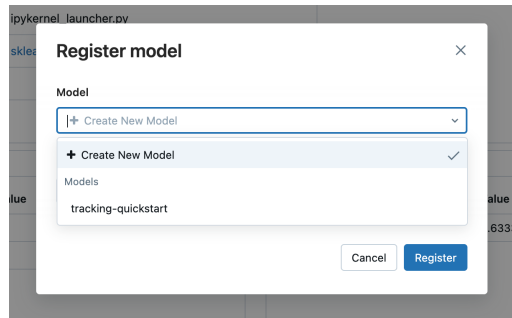
Compare

Version	Registered at	Created by	Tags	Aliases
 Version 1	2024-07-25 16:36:40		Add	Add

MLFlow Registry: Versioning

During registration, it is specified whether the model to be registered should be registered as a new model or whether it should be registered as an existing model with a new version.

Versioning starts with 1 and is automatically incremented whenever a model with an existing name is registered.

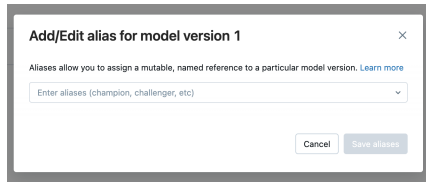


MLFlow Registry: Version Aliases

Model versions can be provided with an alias in the sense of a *mutable named reference*. Aliases are mainly used to

- decouple the executing code from the model version
- represent a deployment status.

In production, for example, a model version with the alias *champion* can be used, while a new version with the alias *challenger* is being worked on simultaneously. Both versions can be used at the same time, e.g., for *A/B Tests* or for deployment patterns such as *Expand and Contract*.



The screenshot shows a modal dialog box titled "Add/Edit alias for model version 1" with a close button (X) in the top right corner. Below the title, there is a descriptive text: "Aliases allow you to assign a mutable, named reference to a particular model version. [Learn more](#)". Underneath this text is a text input field with the placeholder text "Enter aliases (champion, challenger, etc)". At the bottom right of the dialog, there are two buttons: "Cancel" and "Save aliases".

Models can be referenced and thus used in three ways.

```
# by *run id* from Tracking (for UNREGISTERED models)  
logged_model = 'runs:/7180c89f925946...663640810d/iris_model'
```

```
# by model name and version (for REGISTERED models)  
model_uri = "models:/tracking-quickstart/1"  
model_uri = "models:/tracking-quickstart/latest"
```

```
# by model name and version-alias (for REGISTERED models)  
model_uri = "models:/tracking-quickstart@champion"
```

If you have a *model_uri*, the model can be loaded using a variant of the `load_model()` function.

```
# either
loaded_model = mlflow.sklearn.load_model(model_uri)
# or
loaded_model = mlflow.pyfunc.load_model(model_uri)

loaded_model.predict(df)
```

During the development phase, an unregistered model can also be used via the *run_id*.

MLFlow has no usable concept of deployment stages. A model can be *promoted*, but this functionality is limited to copying, renaming, and tagging a model with optional access control.

More info in the official documentation: [Promoting an MLflow Model across environments](#)

Registered models can be *served* with MLFlow:

```
mlflow models serve -m "models:/tracking-quickstart/1"
```

This starts a Python Flask Server in the current environment, provided the necessary dependencies are present.

In addition to the presented high-level *MLFlow Fluent API*, there is a second, lower-level API, the `MlflowClient`.

The MLFlow Client provides a Python CRUD interface to manage experiments, runs, model versions, and registered models.

```
from mlflow import MlflowClient
```

```
client = MlflowClient()  
experiment_id = client.create_experiment("New Experiment")  
client.delete_experiment(experiment_id)
```

In my personal opinion, MLFlow is not the ideal tool. For the course, it has the advantage that we can install it locally for free, and it covers the central components Tracking, Registry, and Model API.

■ Pros

- Free/Open Source and on-premise possible
- Basic functionality easy to understand

■ Cons

- Documentation chaotic, no understandable hierarchy
- Organization confusing (Models, Projects), overlapping responsibilities
- Functionality of central components limited
- Staging functionality confusingly implemented

- Aim
- DVC
- Sacred mit Omnilboard
- Tensorboard
- **Weights and Biases** (WANDB) offers excellent tracking. Ideally in the cloud, but also offers on-premise solutions.
- Azure, AWS, Vertex AI offer tracking, as do Comet, Dagshub, ClearML

- **Weights and Biases** (WANDB) offers a Registry in addition to tracking
- **Hopsworks** offers a Model Registry
- Azure, AWS, Vertex AI offer Model Registries, as do DataRobot, Dataiku, Comet

- Model Platform =

- Model Platform =
 - Deployment API +

- Model Platform =
 - Deployment API +
 - Model Registry +

- Model Platform =
 - Deployment API +
 - Model Registry +
 - Model Server

- Model Platform =
 - Deployment API +
 - Model Registry +
 - Model Server
 - (+ Experiment Tracking)

- Model Platform =
 - Deployment API +
 - Model Registry +
 - Model Server
 - (+ Experiment Tracking)
- Inference can be performed in various ways

- Model Platform =
 - Deployment API +
 - Model Registry +
 - Model Server
 - (+ Experiment Tracking)
- Inference can be performed in various ways
 - Heavily dependent on the use case and infrastructure

- Model Platform =
 - Deployment API +
 - Model Registry +
 - Model Server
 - (+ Experiment Tracking)
- Inference can be performed in various ways
 - Heavily dependent on the use case and infrastructure
- In this course we use MLFlow, but there are also many good alternatives